摄像头显示

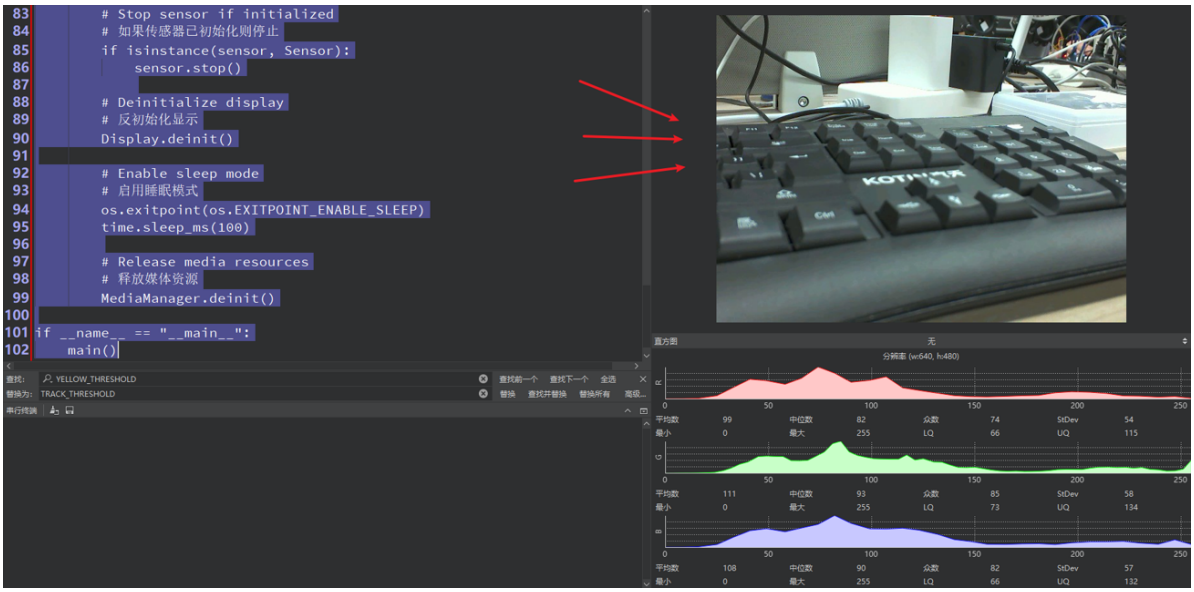
例程实验效果简介
代码讲解
完整代码
摄像头相关API
构造函数
2.1 sensor.reset
2.2 sensor.set_framesize
2.3 sensor.set_pixformat
2.4 sensor.set_hmirror
2.5 sensor.set_vflip
2.6 sensor.run
2.7 sensor.stop
2.8 sensor.snapshot

例程实验效果简介

这一节我们来调用一下摄像头，在IDE的帧缓冲区中显示摄像头捕获到的图像

我们用CanMV IDE打开例程代码，将K230用usb连接到电脑上

点击CanMV IDE左下角的运行按钮，可以看到IDE右上角的帧缓冲区中显示了摄像头捕获到的图像



代码讲解

本节教程的源码文件位于 [源码汇总 / 02.Basic / 17.sensor.py]

完整代码

```
# 导入必要的模块：时间、数学、操作系统、垃圾回收、系统
# (Import necessary modules: time, math, os, garbage collection, system)
import time, math, os, gc, sys

# 导入媒体相关模块：传感器、显示、媒体管理
```

```python
# (Import media-related modules: sensor, display, media manager)
from media.sensor import *
from media.display import *
from media.media import *

# 定义图像宽度和高度常量
# (Define image width and height constants)
WIDTH = 640
HEIGHT = 480

# 初始化传感器变量为空
# (Initialize sensor variable as None)
sensor = None

try:
    # 使用默认配置构造传感器对象，设置指定宽度和高度
    # (Construct a Sensor object with default configuration, setting specified
width and height)
    sensor = Sensor(width = WIDTH, height = HEIGHT, fps=30)

    # 传感器复位
    # (Reset the sensor)
    sensor.reset()

    # 设置水平镜像（当前被注释）
    # (Set horizontal mirror - currently commented out)
    # sensor.set_hmirror(False)

    # 设置垂直翻转（当前被注释）
    # (Set vertical flip - currently commented out)
    # sensor.set_vflip(False)

    # 设置通道0的输出尺寸
    # (Set channel 0 output size)
    sensor.set_framesize(width = WIDTH, height = HEIGHT)

    # 设置通道0的输出格式为RGB565
    # (Set channel 0 output format to RGB565)
    sensor.set_pixformat(Sensor.RGB565)

    # 使用IDE作为输出目标初始化显示
    # (Initialize display using IDE as output target)
    Display.init(Display.ST7701, width = WIDTH, height = HEIGHT, to_ide = True)

    # 初始化媒体管理器
    # (Initialize the media manager)
    MediaManager.init()

    # 启动传感器运行
    # (Start the sensor running)
    sensor.run()

    # 创建时钟对象用于计算帧率
    # (Create a clock object to calculate frames per second)
    fps = time.clock()

    # 主循环
    # (Main loop)
```

```
    while True:
        # 帧率计时器tick
        # (Tick the FPS timer)
        fps.tick()

        # 检查是否应该退出程序
        # (Check if the program should exit)
        os.exitpoint()

        # 从传感器获取一帧图像
        # (Capture a frame from the sensor)
        img = sensor.snapshot()

        # 在屏幕上显示结果图像
        # (Display the resulting image on screen)
        Display.show_image(img)

        # 执行垃圾回收
        # (Perform garbage collection)
        gc.collect()

        # 打印当前帧率
        # (Print the current frames per second)
        print(fps.fps())

except KeyboardInterrupt as e:
    # 捕获键盘中断异常（用户手动停止）
    # (Catch keyboard interrupt exception - user manually stops)
    print(f"user stop")
except BaseException as e:
    # 捕获所有其他异常
    # (Catch all other exceptions)
    print(f"Exception '{e}'")
finally:
    # 无论如何都执行清理工作
    # (Perform cleanup regardless of how the program exits)

    # 停止传感器运行（如果传感器对象存在）
    # (Stop the sensor if the sensor object exists)
    if isinstance(sensor, Sensor):
        sensor.stop()

    # 反初始化显示
    # (Deinitialize the display)
    Display.deinit()

    # 设置退出点，允许进入睡眠模式
    # (Set exit point to enable sleep mode)
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)

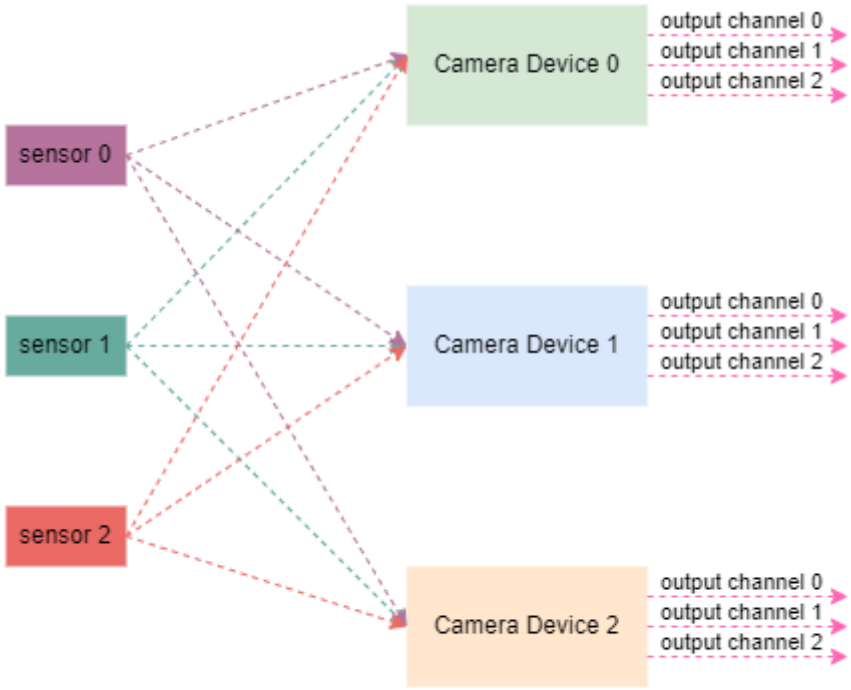
    # 短暂延时100毫秒
    # (Brief delay of 100 milliseconds)
    time.sleep_ms(100)

    # 释放媒体缓冲区
    # (Release media buffer)
    MediaManager.deinit()
```

摄像头相关API

> 以下内容节选自 CanMV Sensor API 章节

CanMV K230 平台的 `sensor` 模块负责图像采集与数据处理。该模块提供了一套高级 API，开发者可以利用这些接口轻松获取不同格式与尺寸的图像，而无需了解底层硬件的具体实现。



K230 CanMV Camera示意图

sensor 0、sensor 1 和 sensor 2 分别代表三个图像输入传感器设备；Camera Device 0、Camera Device 1 和 Camera Device 2 对应相应的图像处理单元；output channel 0、output channel 1 和 output channel 2 表示每个图像处理单元最多支持三个输出通道。通过软件配置，不同的传感器设备可以灵活映射到相应的图像处理单元。

CanMV K230 的 `sensor` 模块最多支持三路图像传感器的同时接入，每一路均可独立完成图像数据的采集、捕获和处理。此外，每个视频通道可并行输出三路图像数据供后端模块进行进一步处理。实际应用中，具体支持的传感器数量、输入分辨率和输出通道数将受限于开发板的硬件配置和内存大小，因此需根据项目需求进行综合评估。

构造函数

描述

通过 `csi id` 和图像传感器类型构建 `Sensor` 对象。

在图像处理应用中，用户通常需要首先创建一个 `Sensor` 对象。CanMV K230 软件可以自动检测内置的图像传感器，无需用户手动指定具体型号，只需设置传感器的最大输出分辨率和帧率。有关支持的图像传感器信息，请参见图像传感器支持列表。如果设定的分辨率或帧率与当前传感器的默认配置不符，系统会自动调整为最优配置，最终的配置可在日志中查看，例如 `use sensor 23, output 640x480@90`。

语法

```
sensor = Sensor(id, [width, height, fps])
```

参数

参数名称	描述	输入/输出	说明
id	`csi` 端口，支持 `0-2`，具体端口请参考硬件原理图	输入	可选，不同型号开发板的默认值不同
width	`sensor` 最大输出图像宽度	输入	可选，默认 `1920`
height	`sensor` 最大输出图像高度	输入	可选，默认 `1080`
fps	`sensor` 最大输出图像帧率	输入	可选，默认 `30`

返回值

返回值	描述
Sensor 对象	传感器对象

举例

```
sensor = Sensor(id=0)
sensor = Sensor(id=0, width=1280, height=720, fps=60)
sensor = Sensor(id=0, width=640, height=480)
```

2.1 sensor.reset

描述

复位 `sensor` 对象。在构造 `Sensor` 对象后，必须调用此函数以继续执行其他操作。

语法

```
sensor.reset()
```

参数

参数名称	描述	输入/输出
无		

返回值

返回值	描述
无	

举例

```
# 初始化 sensor 设备 0 以及传感器 OV5647
sensor.reset()
```

2.2 sensor.set_framesize

描述

设置指定通道的输出图像尺寸。用户可以通过 `framesize` 参数或直接指定 `width` 和 `height` 来配置输出图像尺寸。**宽度会自动对齐到 16 像素宽**。

语法

```
sensor.set_framesize(framesize=FRAME_SIZE_INVALID, chn=CAM_CHN_ID_0, alignment=0,
**kwargs)
```

参数

参数名称	描述	输入/输出
framesize	sensor 输出图像尺寸	输入
chn	sensor 输出通道号	输入
width	输出图像宽度，*kw_arg*	输入
height	输出图像高度，*kw_arg*	输入

返回值

返回值	描述
无	

注意事项

- 输出图像尺寸不得超过图像传感器的实际输出能力。
- 各通道的最大输出图像尺寸受硬件限制。

举例

```
# 配置 sensor 设备 0，输出通道 0，输出图尺寸为 640x480
sensor.set_framesize(chn=CAM_CHN_ID_0, width=640, height=480)

# 配置 sensor 设备 0，输出通道 1，输出图尺寸为 320x240
sensor.set_framesize(chn=CAM_CHN_ID_1, width=320, height=240)
```

2.3 sensor.set_pixformat

描述

配置指定通道的图像传感器输出图像格式。

语法

```
sensor.set_pixformat(pix_format, chn=CAM_CHN_ID_0)
```

参数

参数名称	描述	输入/输出
pix_format	输出图像格式	输入
chn	sensor 输出通道号	输入

返回值

返回值	描述
无	

举例

```
# 配置 sensor 设备 0，输出通道 0，输出 NV12 格式
sensor.set_pixformat(sensor.YUV420SP, chn=CAM_CHN_ID_0)

# 配置 sensor 设备 0，输出通道 1，输出 RGB888 格式
sensor.set_pixformat(sensor.RGB888, chn=CAM_CHN_ID_1)
```

2.4 sensor.set_hmirror

描述

配置图像传感器是否进行水平镜像。

语法

```
sensor.set_hmirror(enable)
```

参数

参数名称	描述	输入/输出
enable	`True` 开启水平镜像功能 `False` 关闭水平镜像功能	输入

返回值

返回值	描述
无	

举例

```
sensor.set_hmirror(True)
```

2.5 sensor.set_vflip

描述

配置图像传感器是否进行垂直翻转。

语法

```
sensor.set_vflip(enable)
```

参数

参数名称	描述	输入/输出
enable	`True` 开启垂直翻转功能 `False` 关闭垂直翻转功能	输入

返回值

返回值	描述
无	

举例

```
sensor.set_vflip(True)
```

2.6 sensor.run

描述

启动图像传感器的输出。必须在调用 `MediaManager.init()` 之前执行此操作。

语法

```
sensor.run()
```

返回值

返回值	描述
无	

注意事项

- 当同时使用多个传感器（最多 3 个）时，仅需其中一个执行 `run` 即可。

举例

```
# 启动 sensor 设备输出数据流
sensor.run()
```

2.7 sensor.stop

描述

停止图像传感器输出。**必须在 `MediaManager.deinit()` 之前调用此方法。**

语法

```
sensor.stop()
```

返回值

返回值	描述
无	

注意事项

- 如果同时使用多个图像传感器（最多 3 个），**每个传感器都需单独调用 `stop`**。

举例

```
# 停止 sensor 设备 0 的数据流输出
sensor.stop()
```

2.8 sensor.snapshot

描述

从指定输出通道中捕获一帧图像数据。

语法

```
sensor.snapshot(chn=CAM_CHN_ID_0)
```

参数

参数名称	描述	输入/输出
chn	sensor 输出通道号	输入

返回值

返回值	描述
image 对象	捕获的图像数据
其他	捕获失败

举例

```
# 从 sensor 设备 0 的通道 0 捕获一帧图像数据
sensor.snapshot()
```